SwiftMart — Sustainable E-Commerce Platform

Group ID: 30

---

1. Project Title & Description

Title: SwiftMart — Sustainable Fashion E-Commerce Platform

Description

SwiftMart is a full-stack, sustainable fashion e-commerce platform built with **Node.js/Express** (backend) and **React + Vite** (frontend). The platform connects multiple stakeholders — **Customers, Sellers, Industry Partners, Managers, and Administrators** — within a single, unified ecosystem that promotes circular fashion and responsible commerce.

Customers can browse verified product catalogs, add items to cart, checkout via Stripe-integrated payments, earn sustainability coins, write product reviews, and donate pre-owned clothing through a structured sell/donate flow with scheduling. Sellers register with identity verification (Aadhaar & GSTN), list products requiring admin approval, manage inventory, view sold-product analytics, and maintain store profiles with Cloudinary-hosted images.

Industry partners (recycling/textile companies) log in to discover donated clothing grouped by fabric-size-usage combinations, add batches to cart, and purchase them through a dedicated checkout — enabling a full circular-economy pipeline.

Managers, created by the Admin, oversee assigned users and sellers for verification workflows with quota-based load balancing. The Admin dashboard provides platform-wide visibility — customer management, seller/product approval pipelines, order tracking with status lifecycle, and revenue analytics.

The backend follows an MVC architecture with JWT cookie-based authentication, role-specific middleware guards, and MongoDB/Mongoose for persistence. The React frontend uses Redux Toolkit for state management, with dedicated API service layers for each role. The platform incorporates an ML-powered clothing classifier to assist in valuation of donated items, making SwiftMart a technologically advanced, environmentally conscious marketplace.

---

2. Domain-Driven Design (DDD) Analysis

2a. Bounded Contexts

The SwiftMart domain is decomposed into **five bounded contexts**, each encapsulating a cohesive set of domain concepts with clear boundaries.

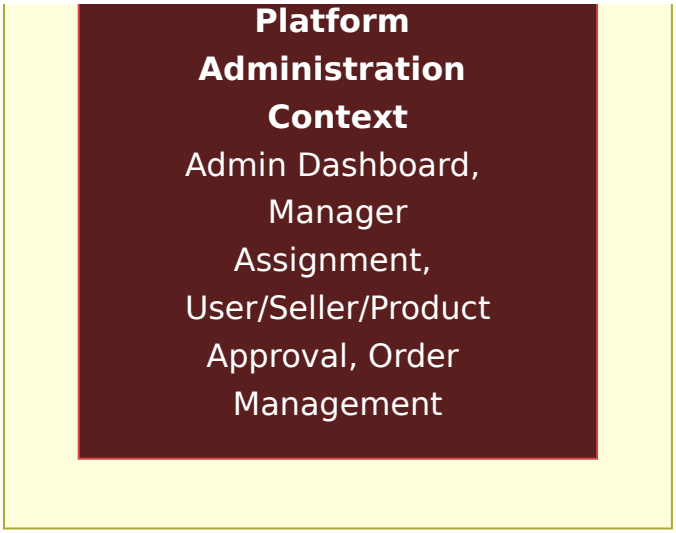
SwiftMart Platform

User Shopping Context
Browsing, Cart, Checkout, Orders, Reviews, Payments

Seller Management Context
Seller Registration, Identity Verification, Product CRUD, Inventory, Sales Analytics

Sustainability & Donation Context
Clothes Donation, Scheduling, Coin Rewards, ML Valuation

Industry Procurement Context
Combination Discovery, Bulk Cart, Industry Checkout, Dashboard Analytics

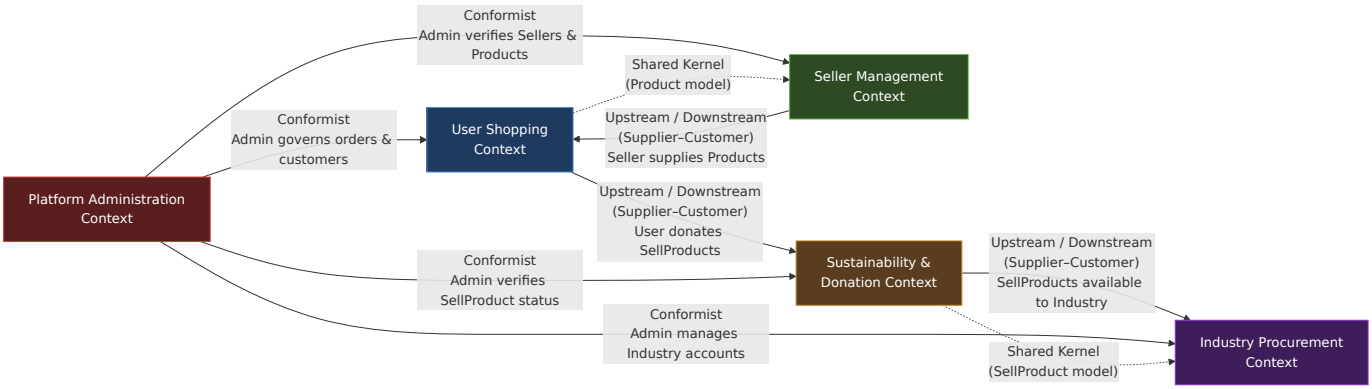


Bounded Context Descriptions

#	Bounded Context	Responsibility	Key Models
1	**User Shopping Context**	End-customer experience: product browsing, cart management, checkout flow, order tracking, payment processing (Stripe), review creation, and profile management	User, Product, Order, Review, UserHistory
2	**Seller Management Context**	Seller onboarding with identity verification, product listing with admin approval workflow, inventory management, sold-product analytics, and store profile management	Seller, Product, SellerHistory
3	**Sustainability & Donation Context**	Users donate pre-owned clothing with fabric/size/condition classification, ML-based valuation, scheduling with time slots, and earning sustainability coins	SellProduct, User (coins)
4	**Industry Procurement Context**	Industry partners (recycling companies) discover donated clothing grouped by combination_id, add batches to cart, checkout (updating SellProduct statuses), and view procurement dashboard	Industry, SellProduct
5	**Platform Administration Context**	Admin dashboard with platform-wide metrics, manager creation with quota-based assignment, user/seller/product approval pipelines, order lifecycle management, and second-hand product oversight	Manager, AssignmentState, Admin (role)

2b. Context Mappings

The diagram below shows relationships between bounded contexts using DDD context mapping patterns.



Context Mapping Table

Upstream Context	Downstream Context	Mapping Pattern	Description
Seller Management	User Shopping	**Supplier–Customer**	Sellers create and supply Products that Users browse, purchase, and review
User Shopping	Sustainability & Donation	**Supplier–Customer**	Users initiate the donation of pre-owned clothing (SellProduct), driving the sustainability workflow
Sustainability & Donation	Industry Procurement	**Supplier–Customer**	Verified SellProducts become available for Industry partners to discover and purchase in bulk
Platform Administration	User Shopping	**Conformist**	Admin manages orders, customers, and platform rules; User Shopping context conforms to admin decisions
Platform Administration	Seller Management	**Conformist**	Admin and Managers verify/reject sellers and products; Seller context conforms
Platform Administration	Sustainability & Donation	**Conformist**	Admin approves/rejects donated items (userStatus) and credits coins; Donation context conforms
Platform Administration	Industry Procurement	**Conformist**	Admin manages industry accounts (CRUD), views industry analytics and procurement rankings
User Shopping ↔ Seller Management	—	**Shared Kernel**	`Product` model is shared between User and Seller contexts
Sustainability & Donation ↔ Industry Procurement	—	**Shared Kernel**	`SellProduct` model is shared between Donation and Industry contexts

2c. Entities, Value Objects, and Services per Bounded Context

Bounded Context 1: User Shopping Context

Type	Name	Key Attributes
Entity	User	_id, firstname, lastname, email, password, coins, managerId (ref Manager), cart[], products[], sellProduct[], reviews[], Address, createdAt, updatedAt
Entity	Product	_id, sellerId, title, price, description, category, image, stock, verified, quantity, reviews[], createdAt, updatedAt
Entity	Order	_id, userId, products[{productId, quantity, price, sellerPrice}], orderStatus, totalAmount, paymentStatus, paymentMethod, paymentProvider, stripeSessionId, coinsUsed, shippingAddress, trackingId, createdAt, updatedAt
Entity	Review	_id, user, product, rating, description
Entity	UserHistory	_id, userId, orders[{orderId, products[], totalAmount, purchaseDate, status}]
Entity	Blog	_id, title, content, author, image, createdAt
Value Object	CartItem	productId, quantity, size
Value Object	ShippingAddress	fullname, street, city, state, pincode, phone
Value Object	UserAddress	plotno, street, city, state, pincode, phone
Value Object	OrderLineItem	productId, quantity, price, sellerPrice
Service	AuthenticationService	User signup, login, Google OAuth (Passport.js), JWT token generation, password hashing (bcrypt), session management
Service	CartService	Add to cart, remove from cart, get cart details, calculate totals
Service	CheckoutService	Create order from cart, apply coin discounts, Stripe session creation

Service	PaymentService	Stripe payment processing, payment status updates, refund handling
Service	ProductDiscoveryService	Product listing, filtering (category, material, gender, size, price range), product details with related products
Service	ReviewService	Create review, associate review with user and product
Service	OrderHistoryService	Track order status, retrieve order history for user

---

Bounded Context 2: Seller Management Context

Type	Name	Key Attributes
Entity	Seller	_id, name, storeName, email, password, gstn, profileImage, products[], address, identityVerification, bankDetails, managerId, createdAt
Entity	Product	_id, sellerId, title, price, description, category, image, stock, verified, quantity, reviews[], createdAt, updatedAt
Entity	SellerHistory	_id, sellerId, productId, action, details, timestamp
Value Object	IdentityVerification	aadharCard, status (Pending/Verified/Rejected)
Value Object	BankDetails	accountNumber, ifscCode, bankName
Value Object	SellerAddress	street, city, state, pincode, country
Service	SellerAuthService	Seller signup with file uploads (profile + Aadhaar to Cloudinary), login, JWT generation
Service	ProductManagementService	Create product with image upload and ML image verification (classifyImage), update product, delete product (cascaded via pre-delete hook), list seller products
Service	SellerProfileService	View/update store profile, account details, and bank information
Service	SalesAnalyticsService	Aggregate sold products by seller, calculate revenue, view sales history

---

Bounded Context 3: Sustainability & Donation Context

Type	Name	Key Attributes
Entity	SellProduct	_id, user_id, items, fabric, size, gender, usageDuration, image{data,contentType}, description, clothesDate, timeSlot, userStatus, adminStatus, estimated_value, combination_id, created_at
Entity	User *(partial — coins only)*	_id, coins, sellProduct[]
Value Object	ClothingCombination	fabric, size, usageDuration → combination_id (e.g., "Cotton_L_3")
Value Object	DonationSchedule	clothesDate, timeSlot (morning/afternoon/evening)
Value Object	ClothingImage	data (Buffer), contentType
Service	DonationSubmissionService	Validate donation form, save SellProduct, generate combination_id, upload image
Service	MLValuationService	TensorFlow gatekeeper + specialist models to classify fabric type and estimate value of donated clothing
Service	CoinRewardService	Calculate and credit sustainability coins to user based on donation valuation
Service	DonationStatusService	Admin review of donations, update userStatus (Pending → Verified/Rejected)

Bounded Context 4: Industry Procurement Context

Type	Name	Key Attributes
Entity	Industry	_id, companyName, email, password, cart[], dashboard[], Address, createdAt, updatedAt
Entity	SellProduct *(read-only reference)*	combination_id, fabric, size, usageDuration, userStatus, adminStatus
Value Object	IndustryCartItem	fabric, size, usageDuration, quantity, amount, combination_id, id

Value Object	DashboardEntry	fabric, size, usageDuration, quantity, amount, combination_id, id, date
Value Object	IndustryAddress	address (string)
Service	IndustryAuthService	Industry signup, login, JWT token generation, profile management
Service	CombinationDiscoveryService	Aggregate SellProducts by combination_id, group by fabric-size-usage, present available quantities
Service	IndustryCartService	Add/remove combination batches to cart, quantity management
Service	IndustryCheckoutService	Checkout flow: Stripe payment session creation, update SellProduct.adminStatus to "Sold", move cart items to dashboard, clear cart
Service	IndustryDashboardService	Fetch purchase history, calculate total procurement amount

---

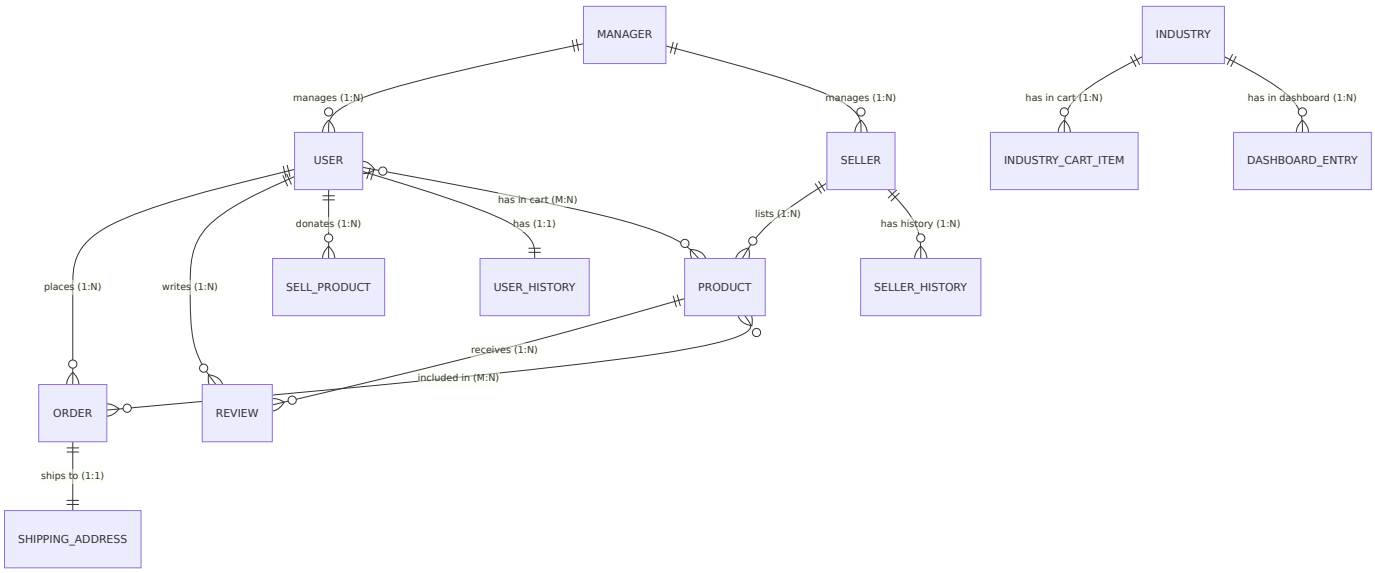
Bounded Context 5: Platform Administration Context

Type	Name	Key Attributes
Entity	Manager	_id, email, password, assignedUserIds[], assignedSellerIds[], pendingUserQuota, pendingSellerQuota, createdAt, updatedAt
Entity	AssignmentState	_id, key, userIndex, sellerIndex
Value Object	ManagerCredentials	email, hashedPassword (SHA-256)
Value Object	AssignmentQuota	pendingUserQuota, pendingSellerQuota
Service	AdminAuthService	Admin login with hardcoded credentials, session management
Service	ManagerManagementService	Create/delete managers, list managers, assign users/sellers with round-robin load balancing
Service	UserApprovalService	List all customers, view customer details, manage customer accounts

Service	SellerApprovalService	Approve/disapprove sellers, verify identity documents
Service	ProductApprovalService	Approve/disapprove products from sellers, view product pipeline
Service	OrderManagementService	View all orders grouped by user, update order status lifecycle (Pending → Processing → Shipped → Delivered)
Service	PlatformDashboardService	Aggregate platform metrics — total users, orders, revenue, seller stats
Service	IndustryManagementService	List all industry partners, view industry details, delete industry accounts

---

2d. Cardinality Ratios



Cardinality Ratio Summary Table

Relationship	Cardinality	Description
User → Order	**1 : N**	One user can place many orders; each order belongs to exactly one user
User → Review	**1 : N**	One user can write many reviews; each review is written by one user
User → SellProduct	**1 : N**	One user can donate many products; each donated product belongs to one user
User → UserHistory	**1 : 1**	One user has exactly one history record tracking all past orders

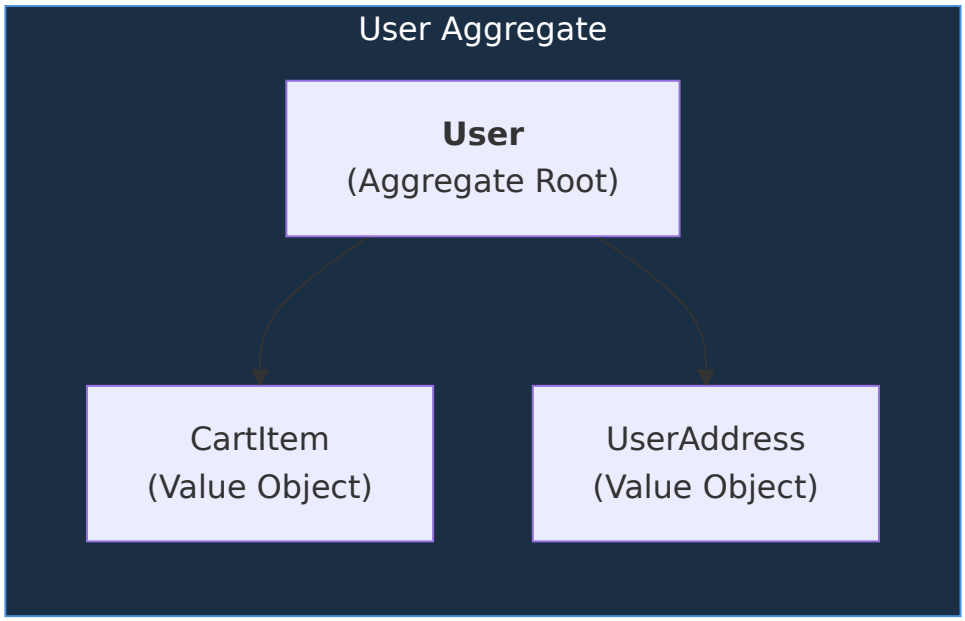
User ↔ Product (Cart)	**M : N**	Many users can have many products in their carts (via CartItem join)
Seller → Product	**1 : N**	One seller lists many products; each product belongs to exactly one seller
Seller → SellerHistory	**1 : N**	One seller has many history entries for product actions (Added/Updated/Deleted/Sold)
Product → Review	**1 : N**	One product can receive many reviews; each review is for one product
Product ↔ Order	**M : N**	Many products can be in many orders (via OrderLineItem join)
Order → ShippingAddress	**1 : 1**	Each order has exactly one embedded shipping address
Industry → IndustryCartItem	**1 : N**	One industry has many items in its cart
Industry → DashboardEntry	**1 : N**	One industry has many purchased entries in its dashboard
Manager → User	**1 : N**	One manager is assigned many users for oversight
Manager → Seller	**1 : N**	One manager is assigned many sellers for verification

---

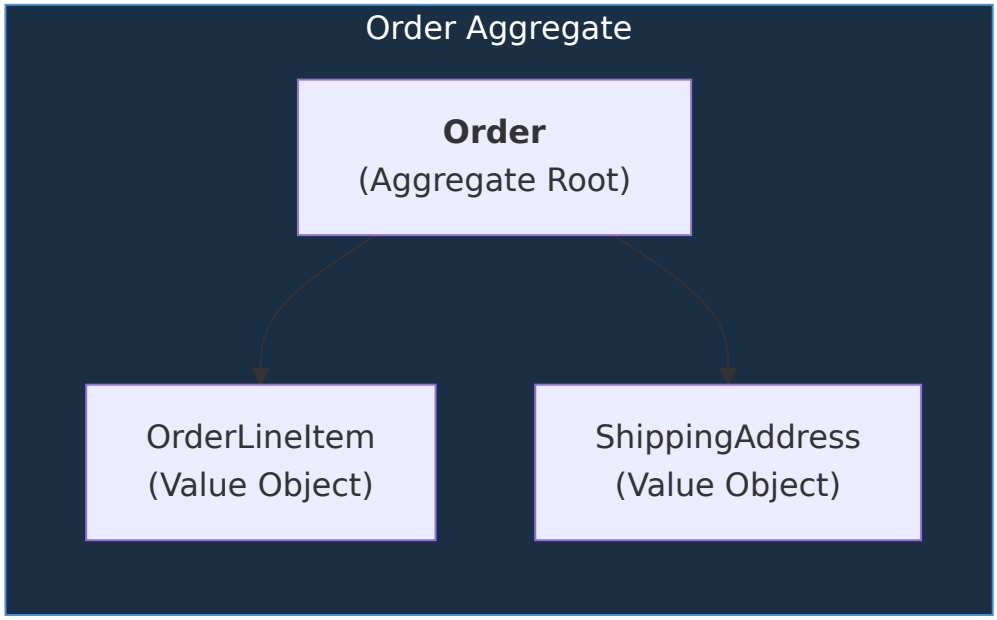
2e. Aggregates

Each aggregate defines a consistency boundary with a single **Aggregate Root** that controls access to its internal entities and value objects.

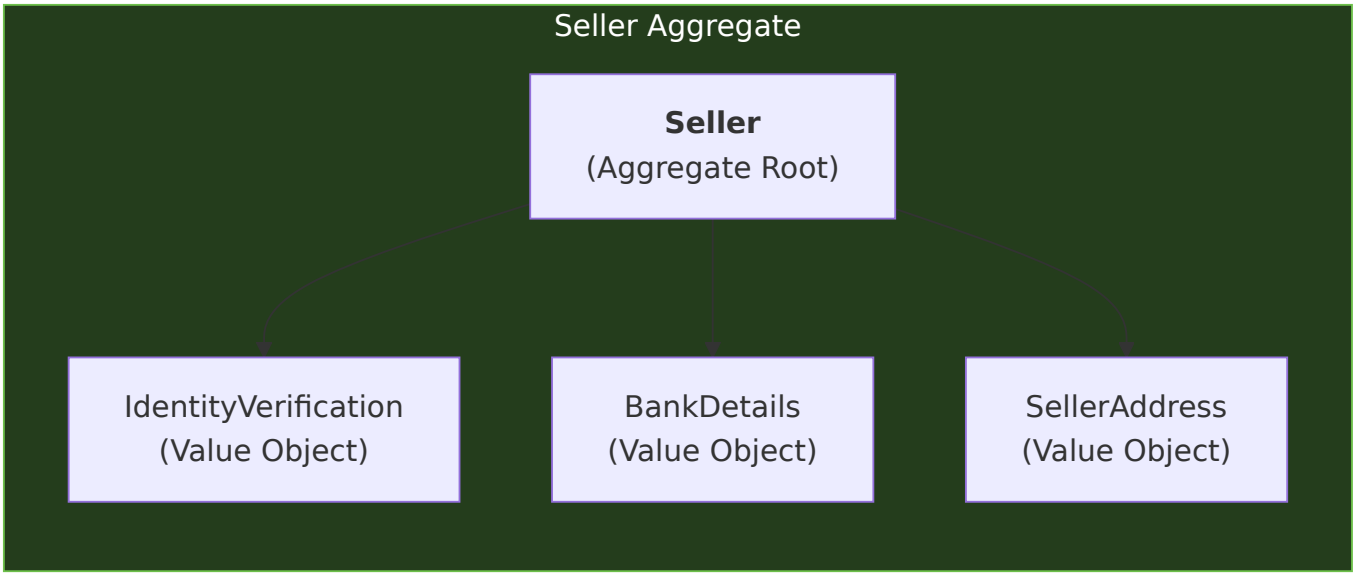
Aggregate 1 — User



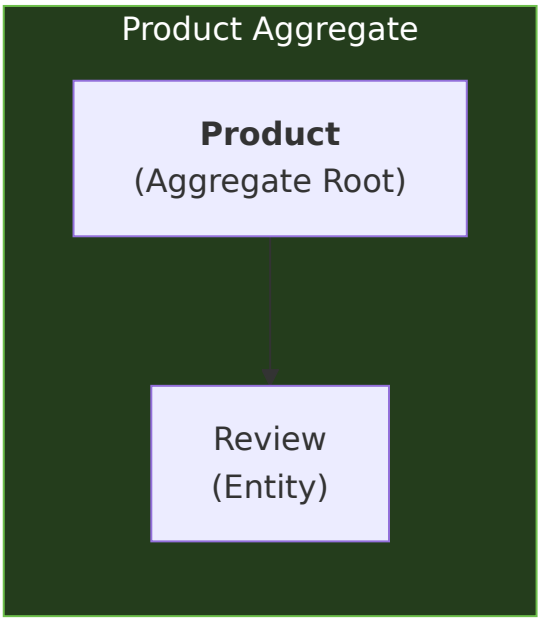
Aggregate 2 — Order



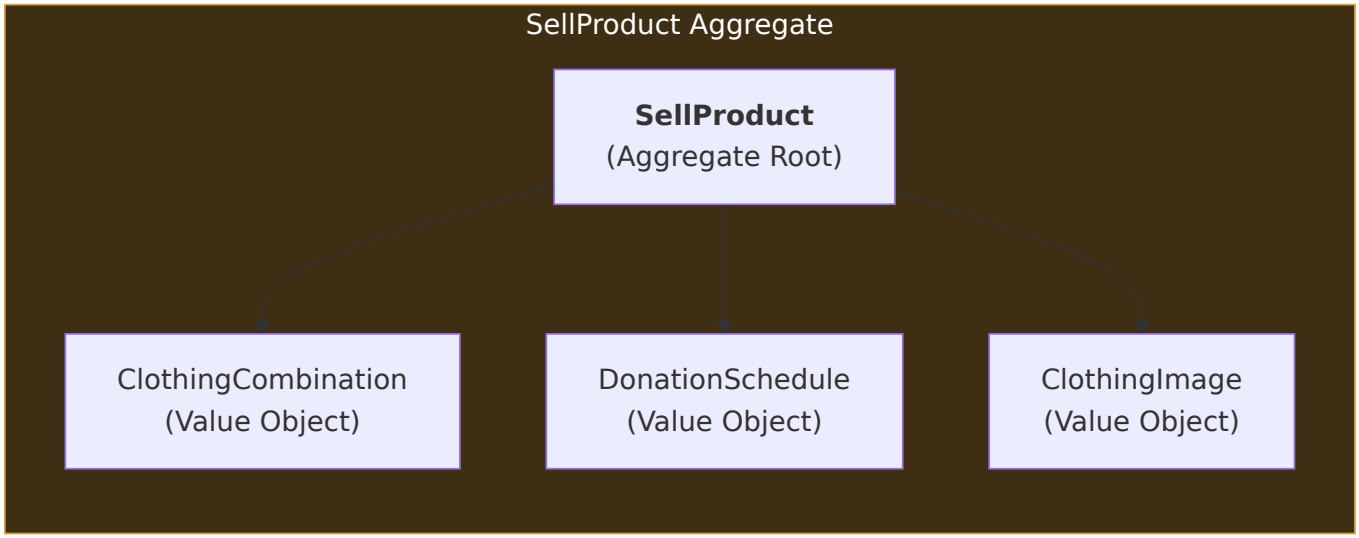
Aggregate 3 — Seller



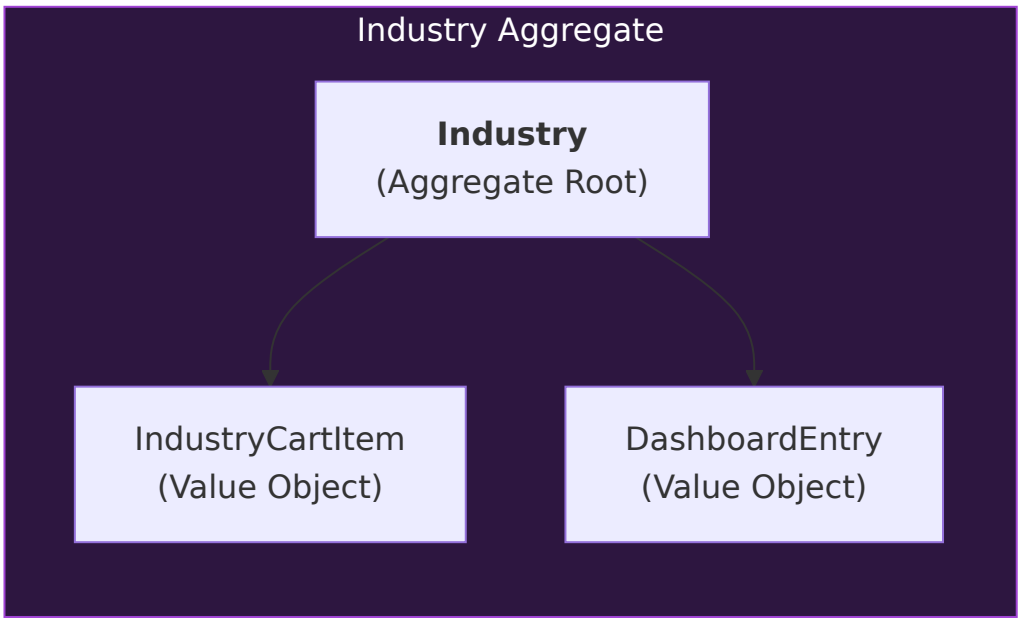
Aggregate 4 — Product



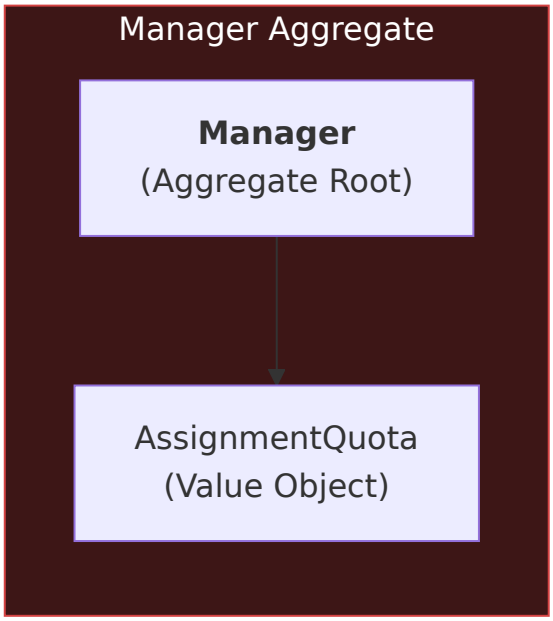
Aggregate 5 — SellProduct



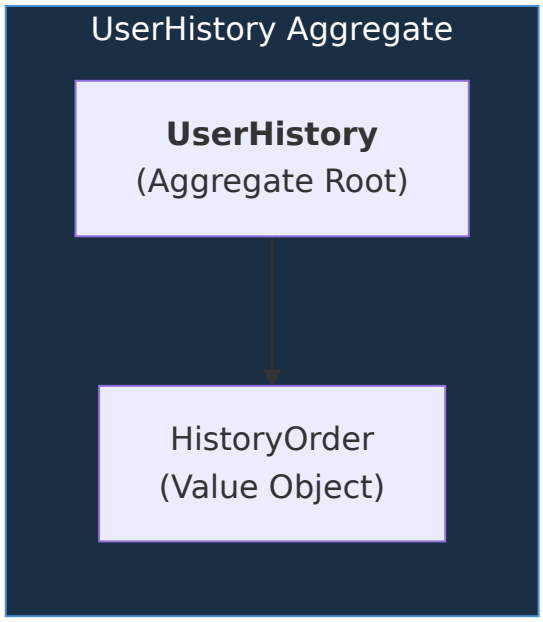
Aggregate 6 — Industry



Aggregate 7 — Manager



Aggregate 8 — UserHistory



Aggregate 9 — SellerHistory

Aggregate Descriptions

#	Aggregate	Root Entity	Internal Entities	Value Objects	Invariants / Rules
1	**User Aggregate**	User	—	CartItem, UserAddress	Cart items reference valid products; coins ⩾ 0; email unique
2	**Order Aggregate**	Order	—	OrderLineItem, ShippingAddress	totalAmount = Σ(quantity × price) − coinsUsed; status transitions: Pending → Processing → Shipped → Delivered/Cancelled/Returned
3	**Seller Aggregate**	Seller	—	IdentityVerification, BankDetails, SellerAddress	GSTN unique; account number unique; identity must be verified before products are approved; cascading delete removes all seller's products

4	**Product Aggregate**	Product	Review	—	Product must be verified by admin/manager before visible in store; stock boolean tracks availability; quantity ⩾ 0
5	**SellProduct Aggregate**	SellProduct	—	ClothingCombination, DonationSchedule, ClothingImage	combination_id = fabric + " " + *size + '"'* + usageDuration; userStatus transitions: Pending → Verified/Rejected; adminStatus transitions: Pending → Sold
6	**Industry Aggregate**	Industry	—	IndustryCartItem, DashboardEntry	Email unique; checkout moves cart → dashboard and sets SellProduct.adminStatus = "Sold"; cart cleared after checkout
7	**Manager Aggregate**	Manager	—	AssignmentQuota	Password SHA-256 hashed; round-robin assignment via AssignmentState; quota tracks pending verifications
8	**UserHistory Aggregate**	UserHistory	—	HistoryOrder	One-to-one with User; immutable order records; stores denormalized product snapshots
9	**SellerHistory Aggregate**	SellerHistory	—	—	Tracks product lifecycle actions (Added/Updated/Deleted/Sold) with timestamps